

Topological Sort — Complete Reference

1. What is it?

A **topological sort** is a linear ordering of the vertices of a **Directed Acyclic Graph (DAG)** such that for every directed edge $u \rightarrow v$, u comes before v in the ordering.

- Only defined for **directed** graphs.
- Only exists if the graph has **no cycle** (a cyclic graph cannot be topologically sorted — if you find a cycle, that's your signal to return "impossible").
- Multiple valid orderings can exist for the same graph.

Think of it as: "given a bunch of *this-before-that* rules, produce one valid sequence that respects all of them."

2. When and why is it used?

Whenever you have tasks/items with **precedence constraints** — some things must happen before others — and you need to:

- Produce a valid execution order, OR
- Detect whether such an order is even possible (cycle detection).

Why: because a naive DFS/BFS traversal doesn't guarantee dependency order — topo sort is the tool purpose-built for "respect these prerequisites."

3. Where is it used (real world)?

- Build systems (Make, Bazel) — compile files in dependency order
- Package managers (npm, pip) — install dependencies before the package
- Course scheduling — prerequisites before advanced courses
- Task/job schedulers — pipeline stages
- Spreadsheet formula evaluation — compute cells in dependency order
- Compilers — symbol/type resolution order

- Deadlock detection (cycle = deadlock)

4. How to identify a topological sort problem (pattern triggers)

You should be able to spot this in under 2 minutes if you see:

Vocabulary signals:

- "prerequisite(s)", "must be completed before", "depends on"
- "build order", "compilation order", "task order", "course schedule"
- "return **any** valid order" / "return **the** order"
- "is it possible to finish all tasks/courses" (cycle detection variant)
- "given pairs (a, b) meaning a must happen before b"

Structural signals (from constraints, not just words):

- Input given as n nodes + a list of **directed** edges/pairs — this is the strongest tell
- Edges represent **precedence**, not distance/weight/cost (if edges have weights and you're asked for shortest path → that's Dijkstra/BFS, not topo sort)
- No mention of cycles being allowed — graph is implied to be (or must be checked as) acyclic
- Asked to detect a cycle in a directed graph → also topo sort (Kahn's algorithm naturally detects cycles)

Yes — constraints alone can tell you. If you see:

`n nodes, edges[] where edges[i] = [a, b] meaning  $a \rightarrow b$  (directed)`

that's almost always topological sort or plain graph traversal. If additionally the question asks for "an order," "is it possible," or "minimum semesters/time to finish," it's topo sort (possibly with BFS-level tracking).

5. Types / variants

Type	Approach	Use when
Kahn's Algorithm (BFS-based)	Track in-degree of each node; repeatedly remove nodes with	Need cycle detection built-in, need level-by-level processing (e.g., "min semesters to finish all

	in-degree 0	courses")
DFS-based	Do a DFS, push node to a stack on post-order (after visiting all children), reverse the stack at the end	Cleaner recursive code, need to detect cycles via recursion-stack (white/grey/black coloring)
All valid orderings	Backtracking over Kahn's algorithm (try each 0-in-degree node)	"Find all possible topological orders"
Lexicographically smallest order	Kahn's + min-heap instead of queue	"Return the smallest order"

6. Rationale / Intuition

- **Kahn's (BFS):** A node can be safely processed only once *all its prerequisites are done*. In-degree = number of unfinished prerequisites. Nodes with in-degree 0 are "ready now." Process them, decrement in-degree of their neighbors (you just removed one of their prerequisites), and any neighbor whose in-degree drops to 0 becomes ready. It's literally peeling the graph layer by layer, like an onion.
- **DFS (post-order + reverse):** To place a node correctly, you must first finish placing everything it depends on. DFS naturally recurses into dependencies first; when a node has no more unvisited dependencies, it's "finished" and safe to prepend to the answer. Reversing the post-order finish sequence gives a valid topo order.
- **Cycle detection:** If Kahn's algorithm finishes and some nodes never reached in-degree 0, there's a cycle (mutual/circular dependency — impossible to schedule). In DFS, a cycle is detected if you revisit a node that's still on the current recursion stack.

7. Time & Space Complexity

For V vertices and E edges:

- **Time:** $O(V + E)$ — every node and edge is processed a constant number of times.
- **Space:** $O(V + E)$ — adjacency list $O(E)$ + in-degree/visited arrays + queue/stack $O(V)$.

This holds for both Kahn's and DFS approaches.

8. Comparison with other graph patterns

Pattern	Purpose	Key data structure	Edge meaning
Topological Sort	Valid order respecting precedence	Queue (Kahn's) / Stack (DFS)	Directed, precedence
Plain BFS/DFS	Traversal, connectivity, shortest path (unweighted)	Queue / recursion stack	Any
Dijkstra / Shortest Path	Minimum cost path	Min-heap (priority queue)	Weighted
Union-Find (DSU)	Connectivity / cycle detection in undirected graphs	Parent array	Undirected
Cycle detection (directed)	Is there a cycle?	Recursion stack coloring, or Kahn's leftover nodes	Directed
Strongly Connected Components (Tarjan/Kosaraju)	Group mutually reachable nodes	DFS + stack	Directed

Key distinguishing question to ask yourself: *"Is this about distance/cost, or about ordering/precedence?"* → cost → shortest path algorithms. Precedence → topological sort.

9. Reusable Java Templates

Template A — Kahn's Algorithm (BFS) — also detects cycles

```
import java.util.*;

public class TopoSortKahn {
    public int[] findOrder(int n, int[][] edges) {
        List<List<Integer>> graph = new ArrayList<>();
        int[] inDegree = new int[n];
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());

        // edges[i] = [a, b] meaning a -> b (a must come before b)
        for (int[] edge : edges) {
            int a = edge[0], b = edge[1];
```

```

        graph.get(a).add(b);
        inDegree[b]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < n; i++) {
        if (inDegree[i] == 0) queue.offer(i);
    }

    int[] order = new int[n];
    int idx = 0;

    while (!queue.isEmpty()) {
        int node = queue.poll();
        order[idx++] = node;

        for (int neighbor : graph.get(node)) {
            inDegree[neighbor]--;
            if (inDegree[neighbor] == 0) {
                queue.offer(neighbor);
            }
        }
    }

    // If idx != n, not all nodes were processed => cycle exists => no valid
    if (idx != n) return new int[0];

    return order;
}
}

```

Template B — DFS-based (post-order + reverse) — with cycle detection

```

import java.util.*;

public class TopoSortDFS {
    private static final int WHITE = 0; // unvisited
    private static final int GREY = 1;  // in current recursion stack
    private static final int BLACK = 2; // fully processed

    public int[] findOrder(int n, int[][] edges) {
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
        for (int[] edge : edges) {
            graph.get(edge[0]).add(edge[1]);
        }
    }
}

```

```

int[] state = new int[n]; // all WHITE by default
Deque<Integer> stack = new ArrayDeque<>();

for (int i = 0; i < n; i++) {
    if (state[i] == WHITE) {
        if (!dfs(i, graph, state, stack)) {
            return new int[0]; // cycle detected
        }
    }
}

int[] order = new int[n];
int idx = 0;
while (!stack.isEmpty()) {
    order[idx++] = stack.pop();
}
return order;
}

// returns false if a cycle is detected
private boolean dfs(int node, List<List<Integer>> graph, int[] state, Deque<I
    state[node] = GREY;

    for (int neighbor : graph.get(node)) {
        if (state[neighbor] == GREY) {
            return false; // back edge -> cycle
        }
        if (state[neighbor] == WHITE) {
            if (!dfs(neighbor, graph, state, stack)) return false;
        }
    }

    state[node] = BLACK;
    stack.push(node); // post-order: push after all children are done
    return true;
}
}

```

Which template to reach for?

- **Kahn's:** when the problem also wants "level" info (e.g., minimum semesters/time — BFS layers = parallel batches), or you prefer iterative code.
- **DFS:** when recursion is natural, or you need lexicographic post-order properties, or the codebase already leans DFS-heavy.

10. Quick mental checklist (use this to decide in under 2 minutes)

1. Is the graph **directed**? (undirected → not topo sort)
2. Do edges represent **precedence/dependency**, not weight/cost?
3. Is the ask "produce an order," "is it possible," "detect a cycle," or "minimum rounds to complete all"?
4. If yes to all three → topological sort. Pick Kahn's if you need cycle detection + layer info; pick DFS if you just need an order and like recursion.